

**[SOFTWARE EDIFICIO MIRADOR]
(DAS) Documento Arquitectura de Software
Versión 1.0**

Identificación de Documento

Identificación	EM1
Proyecto	SOFTWARE EDIFICIO MIRADOR
Versión	1.3
Documento mantenido por	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
Fecha de última revisión	31/03/2026
Fecha de próxima revisión	02/04/2026
Documento aprobado por	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
30/03/2026	1.0	Empezamos el documento DAS y se hablo como grupo que se hace en el ítem 1, logramos hacer 3 contenidos	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
31/03/2026	1.1	Continuamos el documento DAS, y logramos resumir o modificar cosas anteriores para que se entienda mejor	Consuelo Betancur, Eduardo tapia, Álvaro Salazar
31/03/2026	1.2	Terminamos el primer ITEM y continuamos con el segundo ITEM	Consuelo Betancur, Eduardo tapia, Álvaro Salazar

Tabla de Contenidos

1. INTRODUCCIÓN.....	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO.....	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA.....	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA.....	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS.....	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA.....	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD.....	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD).....	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL).....	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

El problema se sitúa en que una comunidad de gran tamaño, 'Edificio Mirador', depende actualmente de un sistema de gestión obsoleto. Esta situación genera una serie de dificultades, entre las que destacan la falta de confianza por parte de los residentes, la ineficiencia en los procesos de cobro y un aumento en los niveles de morosidad, debido principalmente a la inexistencia de un mecanismo eficaz para identificar y monitorear oportunamente a quienes cumplen con sus pagos y a quienes presentan atrasos. Por esto se hace necesaria la implementación de una solución tecnológica automatizada que permita optimizar la gestión administrativa, mejorar la trazabilidad de la información financiera y fortalecer la transparencia.

1.2. Propósito

Implementar un sistema de gestión moderno y automatizado que permita mejorar el control administrativo y financiero del edificio, facilitando la toma de decisiones, el seguimiento de pagos y la gestión eficiente de la información.

1.3. Ámbito

El proyecto se centra en modernizar la administración del edificio "El Mirador", optimizando la gestión de pagos, disminuyendo la morosidad, agilizando procesos internos y fortaleciendo la comunicación con los residentes a través del seguimiento de solicitudes y reclamos, asegurando un funcionamiento más eficiente y transparente.

1.4. Definiciones, acrónimos y abreviaciones

ACRONIMO	DESCRIPCION

1.5. Resumen ejecutivo

El Edificio Mirador enfrenta problemas por un sistema de gestión obsoleto, causando morosidad y falta de transparencia. Se propone implementar una solución tecnológica que modernice la administración, mejore el control de pagos y fortalezca la comunicación con los residentes.

1.6. Arquitectura del sistema

(ej. vista de escenario, vista lógica, vista de desarrollo, vista de proceso, vista física)

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema es una plataforma de gestión que permitirá administrar residentes, registrar pagos de gastos comunes, controlar la morosidad y generar reportes

financieros, la organización y transparencia en la administración del edificio Mirador.

2.2. Objetivos del sistema

Desarrollar implementar un sistema tecnológico que permita gestionar de manera eficiente y automatizada los pagos de las cuotas de gastos comunes, el registro de residentes, el seguimiento de solicitudes y quejas, y la generación de informes, garantizando transparencia, reducción de morosidad y mejora en la comunicación entre la administración y los residentes del edificio "El Mirador".

2.3. Requerimientos funcionales

Gestión de residentes: Registrar, actualizar y eliminar información de residentes. Asociar residentes a departamentos. Mantener historial de cambios de residentes.

Gestión de pagos de gastos comunes: Registrar pagos realizados por los residentes. Generar estados de cuenta individuales. Calcular automáticamente deudas y morosidad. Emitir comprobantes de pago

Control de morosidad: Identificar residentes con pagos pendientes. Generar alertas y notificaciones automáticas. Aplicar intereses o recargos por atraso (si corresponde)

Generación de informes: Generar reportes financieros (ingresos, deudas, morosidad).

2.4. Requerimientos no funcionales

Usabilidad: El sistema debe ser intuitivo tanto para los residentes como para la administración del edificio.

Rendimiento y Disponibilidad: El sistema debe permitir un acceso transparente y a tiempo real de la información dentro del sistema (pagos, etc). Se sugiere que el tiempo de carga del sistema sea inferior a 2 segundos.

Fiabilidad: El sistema debe asegurar precisión en la cobranza y eficiencia de la gestión de recursos para garantizar la transparencia financiera.

Seguridad: El sistema debe garantizar la protección de los datos financieros tanto de los residentes como de la administración.

2.5. Supuestos y dependencias

Acceso a tecnología: La administración deberá contar con un computador y acceso a internet para poder actualizar los datos de pago y cobro en tiempo real.

[illegible]

4.1.6. Especificación de casos de uso

Caso de Uso	[Nombre del Caso de Uso]	Identificador: [Del caso de uso]
Actores	[Listado de los actores que tienen participación en el caso de uso]	
Tipo	[Tipo de caso de uso, primario, secundario, opcional]	
Referencias	[Requerimientos o funcionalidades incluidas en este caso de uso. Casos de uso relacionados.]	
Precondición	[Condiciones sobre el estado del sistema que deben cumplirse para iniciar el caso de uso]	
Postcondición	[Efectos inmediatos que tienen la ejecución del caso de uso sobre el estado del sistema]	
Descripción	[Descripción del caso de uso]	
Resumen	[Resumen de alto nivel del funcionamiento]	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
[Nro. de paso]	[Actor ejecutor o especifica si es el sistema o subsistema]	[Descripción del paso actividad ejecutado]
[Se describe el proceso o secuencia de pasos ejecutadas usando frases cortas] [Cada paso del proceso puede ser ejecutado por los Actores o por el sistema] [Se describe la secuencia de acciones realizadas por los actores y la secuencia de actividades realizada por el sistema como respuesta].		

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
[Número de paso]	[Descripción de la secuencia de acciones alternas para el número de actividad indicado. Debe hacer referencia al número de paso en el curso normal]
[Cada paso descrito en el curso normal, puede tener actividades alternas, según la distribución de escenarios que ocurra en el flujo de procesos, en esta ficha se completa para cada actividad (haciendo referencia a su número) las posibles secuencias alternas]	

4.2. VISTA LÓGICA

4.2.1. Propósito

4.2.2. Diagrama de clases

4.2.3. Descripción diagrama de clases

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

- 4.3.1. Propósito
- 4.3.2. Diagrama de componente
- 4.3.3. Descripción diagrama de componente
- 4.3.4. Diagrama de paquete
- 4.3.5. Descripción diagrama de paquete

4.4. VISTA DE PROCESOS

- 4.4.1. Propósito
- 4.4.2. Diagrama de actividad
- 4.4.3. Descripción diagrama de actividad

4.5. VISTA FÍSICA

- 4.5.1. Propósito
- 4.5.2. Diagrama de despliegue
- 4.5.3. Descripción diagrama de despliegue

5. REQUISITOS DE CALIDAD

5.1. Propósito

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN

5.3. Reglas y criterios de evaluación de calidad

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc.).

Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt